

CampusConnect - Technical Documentation

Table of Contents

1. [Project Overview](#)
 2. [Architecture & Tech Stack](#)
 3. [Backend Architecture](#)
 4. [Frontend Architecture](#)
 5. [Key Features Implementation](#)
 6. [Database Schema](#)
 7. [Authentication & Security](#)
 8. [Real-time Communication](#)
 9. [State Management](#)
 10. [Interview Talking Points](#)
-

Project Overview

CampusConnect is a full-stack social networking platform designed specifically for college students. It enables students to connect, share posts, organize events, and communicate in real-time.

Core Functionalities

- User authentication with JWT tokens
 - Social feed with posts, likes, and comments
 - Real-time messaging system
 - Event management and RSVP
 - Network/Connection management
 - Admin dashboard for moderation
 - Dark mode support
-

Architecture & Tech Stack

Frontend

- **Framework:** React 18 with Vite
- **Routing:** React Router v6
- **Styling:** Tailwind CSS + Custom CSS
- **HTTP Client:** Axios with interceptors
- **Real-time:** Socket.IO Client
- **State Management:** React Context API + useState/useEffect hooks

Backend

- **Runtime:** Node.js with Express.js
- **Database:** MongoDB with Mongoose ODM
- **Authentication:** JWT (Access + Refresh tokens)
- **Real-time:** Socket.IO Server
- **Security:** bcrypt, CORS, cookie-parser
- **Environment:** dotenv for configuration

Development Tools

- **Version Control:** Git & GitHub
 - **Package Manager:** npm
 - **Deployment:** Vercel (Frontend) + Render (Backend)
-

Backend Architecture

Server Structure (`server/`)

```
server/
├─ server.js          # Main entry point
├─ db.js              # MongoDB connection
├─ models/            # Mongoose schemas
│   ├── User.js
│   ├── Post.js
│   ├── Message.js
│   ├── Conversation.js
│   ├── Event.js
│   ├── Connection.js
│   ├── ActivityLog.js
│   └── Student.js
├─ routes/             # API endpoints
│   ├── auth.js
│   ├── posts.js
│   ├── chat.js
│   ├── connections.js
│   ├── events.js
│   └── admin.js
└─ utils/              # Helper functions
```

Server.js - Core Setup

Key Implementation Details:

1. HTTP + Socket.IO Server

```
const httpServer = http.createServer(app);
const io = new SocketIOServer(httpServer, {
  cors: { origin: allowedOrigins, credentials: true }
});
```

2. CORS Configuration

- Dynamic origin validation
- Supports multiple environments (dev, production)
- Credentials enabled for cookie-based auth

3. Socket Authentication Middleware

```
const authenticateSocket = (socket, next) => {
  const token = socket.handshake.auth?.token;
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

```

socket.data.user = decoded;
socket.join(`user:${decoded.id}`);
next();
};

```

Why this approach?

- Ensures only authenticated users can establish WebSocket connections
- Automatically joins user to their personal room for targeted messaging
- Validates JWT before allowing any real-time communication

Database Connection (db.js)

MongoDB Connection Strategy:

- Uses Mongoose for ODM
- Connection pooling for performance
- Environment-based database selection
- Error handling with retry logic

```

export const connectDB = async () => {
  const dbName = process.env.NODE_ENV === 'production'
    ? 'production'
    : 'development';

  await mongoose.connect(process.env.MONGODB_URI, {
    dbName,
    serverSelectionTimeoutMS: 5000
  });
};

```

API Routes

1. Authentication Routes (routes/auth.js)

Endpoints:

- POST /api/auth/signup - User registration
- POST /api/auth/login - User login
- POST /api/auth/refresh - Token refresh
- GET /api/auth/me - Get current user
- PUT /api/auth/profile - Update profile
- GET /api/auth/search - Search users

Key Implementation - Dual Token System:

```

// Login generates both tokens
const accessToken = jwt.sign(payload, JWT_SECRET, { expiresIn: '15m' });
const refreshToken = jwt.sign(payload, JWT_SECRET, { expiresIn: '7d' });

// Refresh token stored in httpOnly cookie
res.cookie('refreshToken', refreshToken, {
  httpOnly: true,

```

```

    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: 7 * 24 * 60 * 60 * 1000
  });

```

Why dual tokens?

- **Access Token:** Short-lived (15min), sent in Authorization header
- **Refresh Token:** Long-lived (7 days), stored in httpOnly cookie
- Prevents XSS attacks (refresh token not accessible via JavaScript)
- Reduces server load (fewer refresh requests)

2. Posts Routes (routes/posts.js)

Endpoints:

- GET /api/posts - Get all posts (paginated)
- POST /api/posts - Create post
- POST /api/posts/:id/like - Like/unlike post
- POST /api/posts/:id/comment - Add comment
- DELETE /api/posts/:id - Delete post
- POST /api/posts/:id/share - Share post via message

Key Feature - Pagination:

```

const page = parseInt(req.query.page) || 1;
const limit = 20;
const skip = (page - 1) * limit;

const posts = await Post.find()
  .populate('author', 'name profile_photo')
  .sort({ createdAt: -1 })
  .skip(skip)
  .limit(limit);

```

3. Chat Routes (routes/chat.js)

Endpoints:

- GET /api/chat/conversations - Get user's conversations
- POST /api/chat/conversations - Create new conversation
- GET /api/chat/conversations/:id/messages - Get messages
- POST /api/chat/conversations/:id/messages - Send message
- POST /api/chat/conversations/:id/read - Mark as read

Real-time Integration:

```

// After saving message to DB, emit via Socket.IO
const io = req.app.get('io');
io.to(`conversation:${conversationId}`).emit('message:new', {
  conversationId,
  message: populatedMessage
});

```

4. Events Routes (routes/events.js)

Endpoints:

- GET /api/events - Get all events
- POST /api/events - Create event
- POST /api/events/:id/rsvp - RSVP to event
- DELETE /api/events/:id/rsvp - Cancel RSVP
- DELETE /api/events/:id - Delete event

5. Connections Routes (routes/connections.js)

Endpoints:

- GET /api/connections - Get user's connections
- GET /api/connections/requests - Get pending requests
- POST /api/connections/request - Send connection request
- POST /api/connections/accept/:id - Accept request
- POST /api/connections/reject/:id - Reject request
- DELETE /api/connections/:id - Remove connection

6. Admin Routes (routes/admin.js)

Endpoints:

- GET /api/admin/stats - Dashboard statistics
- GET /api/admin/users - Get all users (paginated)
- POST /api/admin/users/:id/suspend - Suspend user
- DELETE /api/admin/users/:id - Delete user
- GET /api/admin/activity-logs - Get activity logs
- GET /api/admin/login-history - Get login history

Admin Middleware:

```
const requireAdmin = (req, res, next) => {
  if (!req.user.isAdmin) {
    return res.status(403).json({ error: 'Admin access required' });
  }
  next();
};
```

Frontend Architecture

Project Structure (client/CampusConnect/src/)

```
src/
├─ App.jsx           # Main app component with routing
├─ main.jsx          # Entry point
├─ config.js         # API configuration
├─ api/
│   └─ auth.js       # Axios instances & interceptors
├─ contexts/
│   └─ SocketContext.jsx # Shared Socket.IO connection
```

```

├── context/
│   └── ThemeContext.jsx # Dark mode state
├── pages/
│   ├── Login.jsx
│   ├── Signup.jsx
│   ├── Dashboard.jsx
│   ├── Profile.jsx
│   ├── Messages.jsx
│   ├── Events.jsx
│   ├── Network.jsx
│   └── Admin.jsx
└── components/
    ├── MessagesWidget.jsx
    ├── MessagesPopup.jsx
    └── ConfirmationModal.jsx

```

App.jsx - Routing & Authentication

Key Implementation:

```

export default function App() {
  const [authChecked, setAuthChecked] = useState(false);
  const [isAuthenticated, setIsAuthenticated] = useState(false);

  // Initial auth check on mount
  useEffect(() => {
    (async () => {
      try {
        const res = await refresh();
        if (res.data?.accessToken) {
          setAccessToken(res.data.accessToken);
          setIsAuthenticated(true);
        }
      } catch {
        setIsAuthenticated(false);
      }
      setAuthChecked(true);
    })();
  }, []);

  // Listen for login events
  useEffect(() => {
    const checkAuth = () => {
      const token = getAccessToken();
      setIsAuthenticated(!!token);
    };

    window.addEventListener('tokenUpdated', checkAuth);
    return () => window.removeEventListener('tokenUpdated', checkAuth);
  }, []);

  return (

```

```

    <ThemeProvider>
      {isAuthenticated ? (
        <SocketProvider>
          <Router>{/* Authenticated routes */}</Router>
        </SocketProvider>
      ) : (
        <Router>{/* Public routes */}</Router>
      )}
    </ThemeProvider>
  );
}

```

Why this pattern?

- Prevents flash of unauthenticated content
- Socket connection only for authenticated users
- Automatic re-render on login/logout
- Custom event for same-tab token updates

API Client (api/auth.js)

Axios Interceptor Pattern:

```

const attachInterceptors = (client) => {
  // Request interceptor - attach token
  client.interceptors.request.use((config) => {
    if (accessToken) {
      config.headers.Authorization = `Bearer ${accessToken}`;
    }
    return config;
  });

  // Response interceptor - handle 401 & refresh
  client.interceptors.response.use(
    (response) => response,
    async (error) => {
      if (error.response?.status === 401 && !originalRequest._retry) {
        // Refresh token logic
        const newToken = await refreshAccessToken();
        originalRequest.headers.Authorization = `Bearer ${newToken}`;
        return client(originalRequest);
      }
      return Promise.reject(error);
    }
  );
};

```

Benefits:

- Automatic token attachment to all requests
- Transparent token refresh on 401 errors
- Request queuing during refresh to prevent race conditions

- Centralized error handling

Socket Context (contexts/SocketContext.jsx)

Problem Solved: Previously, multiple components created separate Socket.IO connections, causing:

- 3+ duplicate connections per user
- Unnecessary network overhead
- Potential message synchronization issues

Solution - Centralized Socket Manager:

```
export const SocketProvider = ({ children }) => {
  const socketRef = useRef(null);
  const [isConnected, setIsConnected] = useState(false);

  useEffect(() => {
    const token = getAccessToken();
    if (!token) return;

    socketRef.current = io(API_BASE_URL, {
      auth: { token },
      reconnection: true,
      reconnectionDelay: 1000,
      reconnectionAttempts: 5
    });

    socketRef.current.on('connect', () => setIsConnected(true));
    socketRef.current.on('disconnect', () => setIsConnected(false));

    return () => {
      if (socketRef.current) {
        socketRef.current.disconnect();
      }
    };
  }, []);

  const on = (event, callback) => {
    if (!socketRef.current) return;
    socketRef.current.on(event, callback);

    // Return cleanup function
    return () => {
      if (socketRef.current) {
        socketRef.current.off(event, callback);
      }
    };
  };

  const emit = (event, data) => {
    if (!socketRef.current) return;
    socketRef.current.emit(event, data);
  };
};
```



```

return (
  <SocketContext.Provider value={{ on, emit, isConnected }}>
    {children}
  </SocketContext.Provider>
);
};

```

Usage in Components:

```

const Messages = () => {
  const { on, emit } = useSocket();

  useEffect(() => {
    const unsubscribe = on('message:new', (data) => {
      setMessages(prev => [...prev, data.message]);
    });

    return () => unsubscribe();
  }, [on]);

  const sendMessage = () => {
    emit('join:conversation', conversationId);
  };
};

```

Benefits:

- Single socket connection shared across app
- Proper cleanup prevents memory leaks
- Automatic reconnection on disconnect
- Type-safe event handling

Key Features Implementation

1. Dashboard Feed

File: `pages/Dashboard.jsx`

Features:

- Infinite scroll pagination
- Post creation with image upload
- Like/unlike functionality
- Comment system
- Post sharing via messages
- Dark mode support

Key Implementation - Post Creation:

```

const handleCreatePost = async (e) => {
  e.preventDefault();

```

```

const formData = {
  content: newPostContent,
  image: newPostImage // Base64 encoded
};

const response = await PostsAPI.post('/', formData);
setPosts([response.data.post, ...posts]);
setNewPostContent('');
setNewPostImage(null);
};

```

Image Upload Strategy:

- Client-side image compression
- Base64 encoding for simplicity
- 50MB limit on server
- Stored directly in MongoDB (suitable for small-scale)

For Production: Would use cloud storage (AWS S3, Cloudinary) with signed URLs

2. Real-time Messaging

File: `pages/Messages.jsx`

Architecture:

- Conversation-based messaging
- Real-time message delivery via Socket.IO
- Read receipts
- Online status indicators
- Message search

Message Flow:

1. Sending a Message:

```

const sendMessage = async () => {
  const response = await
  fetch(`${API_BASE_URL}/api/chat/conversations/${conversationId}/messages`, {
    method: 'POST',
    body: JSON.stringify({ text: newMessage })
  });

  // Server emits via Socket.IO to all participants
};

```

2. Receiving Messages:

```

useEffect(() => {
  const unsubscribe = on('message:new', (data) => {
    if (selectedConversation?.id === data.conversationId) {
      setMessages(prev => {

```

```

    const exists = prev.some(m => m.id === data.message.id);
    if (exists) return prev; // Prevent duplicates
    return [...prev, data.message];
  });
}
fetchConversations(); // Update conversation list
});

return () => unsubscribe();
}, [on]);

```

Why this approach?

- Optimistic UI updates
- Duplicate prevention
- Conversation list stays in sync
- Works even if socket temporarily disconnects

3. Event Management

File: `pages/Events.jsx`

Features:

- Create events with details (title, description, date, location, type)
- RSVP system
- Filter by event type
- View attendees
- Edit/delete own events

RSVP Implementation:

```

const handleRSVP = async (eventId) => {
  const event = events.find(e => e.id === eventId);
  const isAttending = event.attendees.some(a => a.id === currentUser.id);

  if (isAttending) {
    await EventsAPI.delete(`/${eventId}/rsvp`);
  } else {
    await EventsAPI.post(`/${eventId}/rsvp`);
  }

  fetchEvents(); // Refresh to show updated attendee count
};

```

4. Network/Connections

File: `pages/Network.jsx`

Features:

- Send connection requests
- Accept/reject requests
- View all connections

- Search users
- Connection status indicators

Connection States:

- `null` - No connection
- `pending` - Request sent/received
- `accepted` - Connected

State Management:

```
const getConnectionStatus = (userId) => {
  const connection = connections.find(c =>
    c.user1.id === userId || c.user2.id === userId
  );

  if (!connection) return null;

  if (connection.status === 'accepted') return 'connected';
  if (connection.user1.id === currentUser.id) return 'sent';
  return 'received';
};
```

5. Admin Dashboard

File: `pages/Admin.jsx`

Features:

- User management (suspend, delete, edit)
- Content moderation (delete posts, events)
- Activity logs
- Login history
- Failed login attempts monitoring
- Statistics dashboard

Security:

- Admin-only routes protected by middleware
- Confirmation modals for destructive actions
- Activity logging for audit trail

Key Implementation - Confirmation Modal:

```
const handleDeleteUser = (userId) => {
  setModalConfig({
    isOpen: true,
    type: 'delete_user',
    targetId: userId,
    title: "Delete User",
    message: "Are you sure? This action cannot be undone.",
    isDanger: true
  });
};
```

```
const handleConfirmAction = async () => {
  const { type, targetId } = modalConfig;

  if (type === 'delete_user') {
    await AdminAPI.delete(`/users/${targetId}`);
    fetchUsers();
  }

  setModalConfig({ ...modalConfig, isOpen: false });
};
```

Database Schema

User Model

```
{
  name: String,
  email: String (unique, indexed),
  password: String (hashed with bcrypt),
  bio: String,
  profile_photo: String (base64 or URL),
  portfolio_url: String,
  linkedin_url: String,
  github_url: String,
  leetcode_url: String,
  isAdmin: Boolean,
  suspended: Boolean,
  lastActive: Date,
  createdAt: Date,
  updatedAt: Date
}
```

Post Model

```
{
  content: String,
  image: String,
  author: ObjectId (ref: User),
  likes: [ObjectId] (ref: User),
  comments: [{
    user: ObjectId (ref: User),
    text: String,
    createdAt: Date
  }],
  createdAt: Date,
  updatedAt: Date
}
```

Message Model

```
{
  conversation: ObjectId (ref: Conversation),
  sender: ObjectId (ref: User),
  text: String,
  post: ObjectId (ref: Post), // For shared posts
  read: Boolean,
  createdAt: Date
}
```

Conversation Model

```
{
  participants: [ObjectId] (ref: User),
  lastMessage: ObjectId (ref: Message),
  lastMessageAt: Date,
  createdAt: Date
}
```

Event Model

```
{
  title: String,
  description: String,
  date: Date,
  location: String,
  type: String (enum: ['workshop', 'seminar', 'sports', 'cultural', 'other']),
  organizer: ObjectId (ref: User),
  attendees: [ObjectId] (ref: User),
  image: String,
  createdAt: Date
}
```

Connection Model

```
{
  user1: ObjectId (ref: User),
  user2: ObjectId (ref: User),
  status: String (enum: ['pending', 'accepted', 'rejected']),
  createdAt: Date
}
```

ActivityLog Model

```
{
  user: ObjectId (ref: User),
  action: String,
  details: Object,
  ipAddress: String,
  userAgent: String,
  createdAt: Date
}
```

Authentication & Security

JWT Token Strategy

Access Token:

- Lifetime: 15 minutes
- Storage: Memory (JavaScript variable)
- Purpose: API authentication
- Payload: { id, email, name, isAdmin }

Refresh Token:

- Lifetime: 7 days
- Storage: httpOnly cookie
- Purpose: Generate new access tokens
- Security: Not accessible via JavaScript (XSS protection)

Token Refresh Flow

1. User logs in
↓
2. Server generates access + refresh tokens
↓
3. Access token sent in response body
Refresh token set as httpOnly cookie
↓
4. Client stores access token in memory
↓
5. On API request, access token in Authorization header
↓
6. If 401 error, request new access token using refresh token
↓
7. Server validates refresh token from cookie
↓
8. New access token generated and returned
↓
9. Original request retried with new token

Password Security

```
// Hashing on signup
const hashedPassword = await bcrypt.hash(password, 10);

// Verification on login
const isValid = await bcrypt.compare(password, user.password);
```

CORS Configuration

```
const corsOptions = {
  origin: (origin, callback) => {
    if (allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('CORS policy violation'));
    }
  },
  credentials: true, // Allow cookies
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS', 'PATCH']
};
```

Input Validation

- Email format validation
 - Password strength requirements
 - Content length limits
 - SQL injection prevention (MongoDB parameterized queries)
 - XSS prevention (React auto-escapes)
-

Real-time Communication

Socket.IO Events

Client → Server:

- `join:conversation` - Join a conversation room

Server → Client:

- `message:new` - New message in conversation
- `conversation:update` - Conversation metadata changed
- `connect` - Connection established
- `disconnect` - Connection lost

Room-based Broadcasting

```
// User joins conversation
socket.join(`conversation:${conversationId}`);

// Server broadcasts to all in room
io.to(`conversation:${conversationId}`).emit('message:new', data);
```



```
// User-specific room
socket.join(`user:${userId}`);
io.to(`user:${userId}`).emit('notification', data);
```

Benefits:

- Targeted message delivery
 - Scalable architecture
 - No unnecessary broadcasts
 - Easy to add features (typing indicators, etc.)
-

State Management

Context API Usage

1. ThemeContext - Dark mode state

```
const [isDarkMode, setIsDarkMode] = useState(
  localStorage.getItem('darkMode') === 'true'
);
```

2. SocketContext - Shared socket connection

```
const { on, emit, isConnected } = useSocket();
```

Local State Patterns

1. Pagination State:

```
const [posts, setPosts] = useState([]);
const [page, setPage] = useState(1);
const [hasMore, setHasMore] = useState(true);
```

2. Form State:

```
const [form, setForm] = useState({ email: '', password: '' });
const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });
```

3. Modal State:

```
const [modalConfig, setModalConfig] = useState({
  isOpen: false,
  type: null,
  data: null
});
```

Interview Talking Points

1. "Tell me about your project architecture"

Answer: "CampusConnect is a full-stack MERN application with a clear separation of concerns. The backend is a RESTful API built with Express and MongoDB, handling authentication, data persistence, and business logic. The frontend is a React SPA that communicates with the backend via Axios and Socket.IO."

I implemented a dual-token authentication system using JWT - access tokens for API requests and refresh tokens stored in httpOnly cookies for security. For real-time features, I integrated Socket.IO with a centralized context provider to avoid duplicate connections.

The architecture follows best practices: modular route handlers, middleware for authentication and authorization, Mongoose models for data validation, and React hooks for state management."

2. "How did you handle authentication?"

Answer: "I implemented a secure dual-token JWT system. When a user logs in, the server generates two tokens:

1. **Access Token** (15min lifespan) - sent in the response body and stored in memory on the client. Used for API authentication via Authorization headers.
2. **Refresh Token** (7 days lifespan) - stored in an httpOnly cookie, which prevents XSS attacks since JavaScript can't access it.

When the access token expires, the client automatically requests a new one using the refresh token. I implemented this using Axios interceptors that catch 401 errors, refresh the token, and retry the original request - all transparent to the user.

I also added a request queue to handle multiple simultaneous requests during token refresh, preventing race conditions."

3. "Explain your real-time messaging implementation"

Answer: "The messaging system uses Socket.IO for real-time communication. Initially, I had an issue where multiple components were creating separate socket connections, causing 3+ connections per user.

I solved this by creating a SocketContext that provides a single, shared socket connection to the entire app. The context exposes `on()` and `emit()` methods with proper cleanup to prevent memory leaks.

When a user sends a message, it's first saved to MongoDB via a REST API, then the server broadcasts it to all participants in that conversation room using `io.to(conversationId).emit()`. The client listens for `message:new` events and updates the UI in real-time.

I also implemented duplicate prevention by checking message IDs before adding to state, and the conversation list automatically updates when new messages arrive."

4. "How did you optimize performance?"

Answer: "Several optimizations:

1. **Pagination** - Posts and admin data are paginated to reduce initial load time and memory usage.
2. **Single Socket Connection** - Centralized socket management reduced network overhead by 66%.

3. **Axios Interceptors** - Automatic token refresh prevents unnecessary re-authentication flows.
4. **React Optimization** - Used `useCallback` and `useMemo` where appropriate, proper dependency arrays in `useEffect` to prevent unnecessary re-renders.
5. **Database Indexing** - Added indexes on frequently queried fields like email and user IDs.
6. **Connection Pooling** - MongoDB connection pooling for efficient database access.

For production, I would add:

- Image CDN (Cloudinary/S3) instead of base64
- Redis for caching
- Code splitting and lazy loading
- Service workers for offline support"

5. "What challenges did you face and how did you solve them?"

Answer: "Challenge 1: Login Redirect Issue After successful login, users were stuck on the login page. The issue was that `App.jsx` checked `getAccessToken()` directly, which doesn't trigger a re-render.

Solution: I added an `isAuthenticated` state and a custom event listener for `tokenUpdated`. When login succeeds, it dispatches this event, triggering `App.jsx` to re-render with authenticated routes.

Challenge 2: Duplicate Socket Connections Multiple components were creating separate `Socket.IO` connections, causing 3+ connections per user.

Solution: Created a `SocketContext` that initializes one connection and provides it to all components via `React Context`. This reduced connections from 3+ to 1.

Challenge 3: Token Refresh Race Conditions Multiple API calls during token refresh caused issues.

Solution: Implemented a request queue in the `Axios` interceptor. When refreshing, subsequent requests are queued and replayed after the new token arrives."

6. "How would you scale this application?"

Answer: "For scaling to thousands of users:

Backend:

- Horizontal scaling with load balancers (NGINX)
- Redis for session management and caching
- Database sharding for large datasets
- Message queue (RabbitMQ/Kafka) for async tasks
- Microservices architecture for independent scaling

Frontend:

- CDN for static assets
- Code splitting and lazy loading
- Virtual scrolling for large lists
- Service workers for offline functionality
- Progressive Web App (PWA)

Real-time:

- `Socket.IO` Redis adapter for multi-server support

- Dedicated WebSocket servers
- Message broker for cross-server communication

Database:

- Read replicas for query distribution
- Caching layer (Redis)
- Cloud storage (S3) for media files
- Database indexing optimization

Monitoring:

- Application Performance Monitoring (APM)
- Error tracking (Sentry)
- Analytics (Google Analytics)
- Server monitoring (Prometheus/Grafana)"

7. "Explain your database schema design"

Answer: "I designed the schema following MongoDB best practices:

User Model - Core entity with profile information, authentication data, and social links.

Post Model - Embedded comments array for performance (fewer queries), referenced author for data consistency.

Message/Conversation Models - Separate models for scalability. Conversations track participants and last message for efficient list rendering.

Connection Model - Stores bidirectional relationships with status tracking (pending/accepted).

Event Model - Referenced organizer and attendees array for RSVP functionality.

ActivityLog Model - Audit trail for admin actions and security monitoring.

I used references (ObjectId) for entities that change frequently and embedding for data that's read together. Indexes on email, user IDs, and timestamps for query performance."

8. "What security measures did you implement?"

Answer: "Multiple layers of security:

1. **Authentication:** JWT tokens with short expiration, refresh tokens in httpOnly cookies
2. **Password Security:** bcrypt hashing with salt rounds
3. **CORS:** Strict origin validation, credentials enabled only for trusted origins
4. **Input Validation:** Email format, password strength, content length limits
5. **SQL Injection:** MongoDB parameterized queries prevent injection
6. **XSS Prevention:** React auto-escapes, no dangerouslySetInnerHTML
7. **Authorization:** Middleware checks for admin routes, user ownership verification
8. **Activity Logging:** Track all admin actions and failed login attempts
9. **Rate Limiting:** (Would add) Prevent brute force attacks
10. **HTTPS:** (Production) Encrypted communication"

9. "How did you handle errors?"

Answer: "Comprehensive error handling at multiple levels:

Backend:

- Try-catch blocks in all async functions
- Centralized error middleware
- Specific error messages for debugging
- HTTP status codes (400, 401, 403, 404, 500)
- Activity logging for critical errors

Frontend:

- Axios interceptors catch network errors
- Try-catch in async operations
- User-friendly error messages
- Fallback UI for failed states
- Console logging for debugging

Socket.IO:

- Connection error handlers
- Reconnection logic with exponential backoff
- Graceful degradation if WebSocket fails

For production, I would add:

- Error tracking service (Sentry)
- Error boundaries in React
- Retry logic for transient failures
- User error reporting mechanism"

10. "What would you improve given more time?"

Answer: "Several enhancements:

Features:

- Video/voice calling (WebRTC)
- File sharing in messages
- Notifications system (push notifications)
- Email verification
- Password reset flow
- Two-factor authentication
- User blocking/reporting
- Advanced search with filters
- Hashtags and mentions

Technical:

- TypeScript for type safety
- Unit and integration tests (Jest, React Testing Library)
- E2E tests (Cypress)
- CI/CD pipeline (GitHub Actions)
- Docker containerization
- Kubernetes orchestration
- GraphQL for flexible queries
- Server-side rendering (Next.js)
- Mobile app (React Native)

Performance:

- Image optimization and CDN
- Database query optimization
- Caching strategy (Redis)
- Lazy loading and code splitting
- Virtual scrolling
- Debouncing search inputs

UX:

- Skeleton loaders
 - Optimistic UI updates
 - Offline support
 - Accessibility improvements (ARIA labels)
 - Internationalization (i18n)"
-

Deployment

Frontend (Vercel)

- Automatic deployments from GitHub
- Environment variables for API_BASE_URL
- Build command: `npm run build`
- Output directory: `dist`

Backend (Render)

- Node.js environment
- Environment variables: MongoDB URI, JWT secrets, CORS origins
- Start command: `node server.js`
- Auto-deploy on git push

Environment Variables

Backend (.env):

```
MONGODB_URI=mongodb+srv://...  
JWT_SECRET=your-secret-key  
PORT=5001  
NODE_ENV=production  
CLIENT_ORIGIN=https://your-frontend.vercel.app
```

Frontend (.env):

```
VITE_API_URL=https://your-backend.render.com
```

Conclusion

This project demonstrates proficiency in:

- Full-stack development (MERN stack)
- RESTful API design
- Real-time communication (WebSockets)
- Authentication & authorization

- Database design (MongoDB)
- State management (React Context)
- Security best practices
- Modern JavaScript (ES6+)
- Git version control
- Deployment (Vercel, Render)

The codebase is production-ready with proper error handling, security measures, and scalable architecture. It showcases the ability to build complex, real-world applications from scratch.